

Universitatea Ovidius din Constanta
Facultatea de Matematica si Informatica
Specializarea Informatica

Ingineria sistemelor software

Curs 2

Conf. univ. dr. Crenguta M. Puchianu
2020



Partea I. Modele



Modele

- Un **model** este o descriere (sau specificație) rezultată în urma aplicării mecanismului de abstractizare asupra unui domeniu sau a unui sistem. Un domeniu este o porțiune limitată a lumii reale sau imaginată.
- Obiective:
 - Oferă o bază pentru construirea sistemelor.
 - Folosind rafinarea, putem construi modele pe nivele diferite de abstractizare.
 - Permit obținerea unei soluții bune din punct de vedere calitativ.
 - Furnizează produse de lucru reutilizabile.
 - Previn efectuarea unei modificări și facilitează restructurarea.
 - Furnizează o bază de comunicare între membri echipei de dezvoltare, respectiv între echipă și clienți.



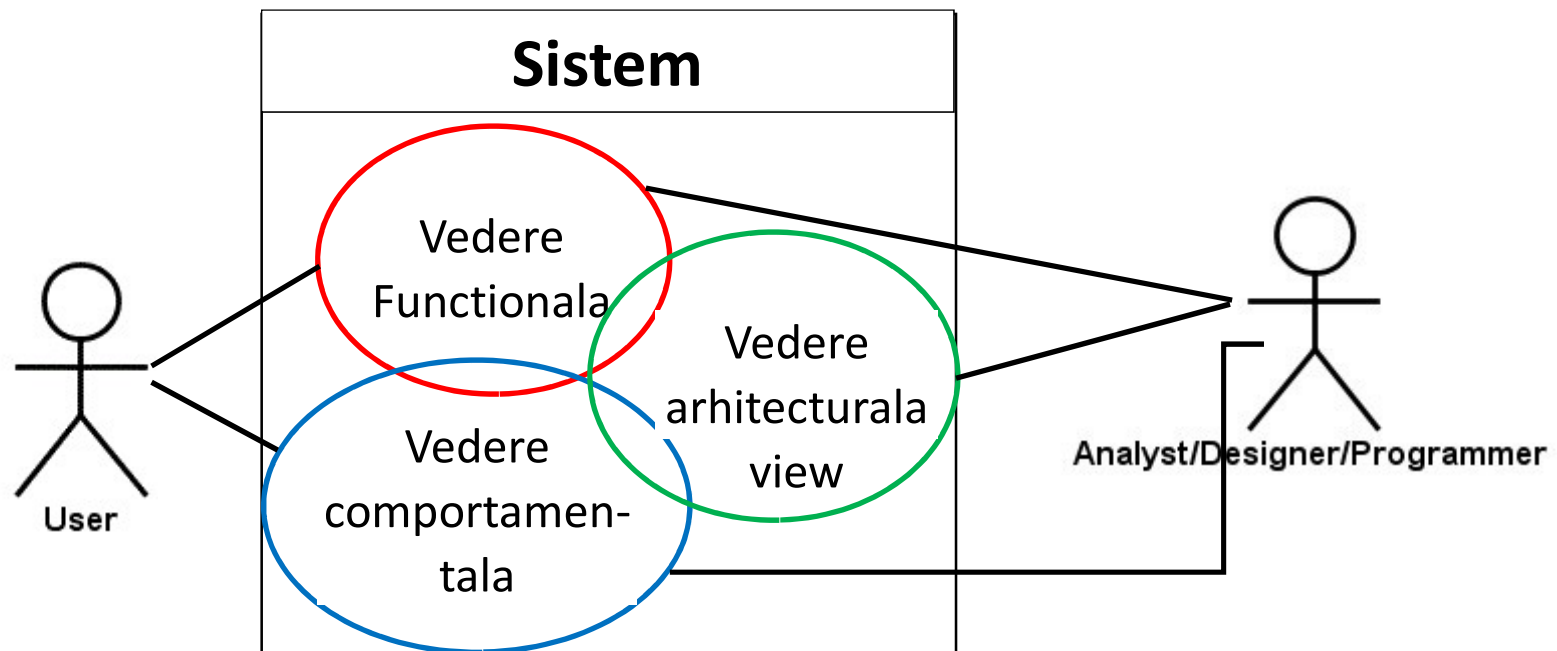
Mecanisme de construire a modelelor

- **Conceptualizare** = capacitatea de abstractizare a unei clase de obiecte care sunt apoi încorporate într-o imagine sau într-un concept, și de asemenea, capacitatea de a sesiza proprietățile distinctive ale unei clase de obiecte.
- **Concept** = entitate cognitivă și semantică ce permite organizarea și interpretarea entităților concrete sau abstracte.
- **Abstractizare** = considerarea anumitor concepte cu anumite proprietati
- **Rafinare** = adaugarea altor concepte si/sau altor proprietati conceptelor existente



Puncte de vedere si vederi

- **Punct de vedere** = tehnică de abstractizare folosind o mulțime de concepte arhitecturale și reguli de structurare cu scopul de a ne concentra pe o mulțime particulară de aspecte de interes din acel sistem.
- **Vedere** = model a unui sistem dintr-un punct de vedere ales.



Limbajul UML (Unified Modeling Language)

- Limbaj semi-formal standardizat in 1997
- Realizat de catre Booch, Rumbaugh si Jacobson pe baza: OMT, OOSE si Booch method
- “este folosit pentru a specifica, vizualiza, construi și documenta artefactele unui sistem software”
- **Artefactele** capteaza cunostintele despre elementele componente ale unui proces initiat sau realizat de un sistem.
- Limbajul poate fi folosit pentru specificarea oricarui tip de sistem (software sau nu), domeniu sau proces (business sau software)
- UML permite captarea si prezentarea cunostintelor pentru a facilita comunicarea intre dezvoltatori si intre dezvoltatori si beneficiari.
- URL: <http://uml.org/>



Vederi si diagrame construite cu UML

Vedere	Diagrame UML
Vederea cazurilor de utilizare	Diagrama cazurilor de utilizare
Vederea statica	Diagrama de clase
Vederea de implementare	Diagrama de componente
Vederea de repartitie	Diagrama de repartitie
Vederea de interactiuni	Diagrama de secventa
	Diagrama de colaborare
Vederea masinii de stari	Masini de stari
Vederea de activitati	Diagrama de activitati



Partea a II-a. Modele ale procesului software

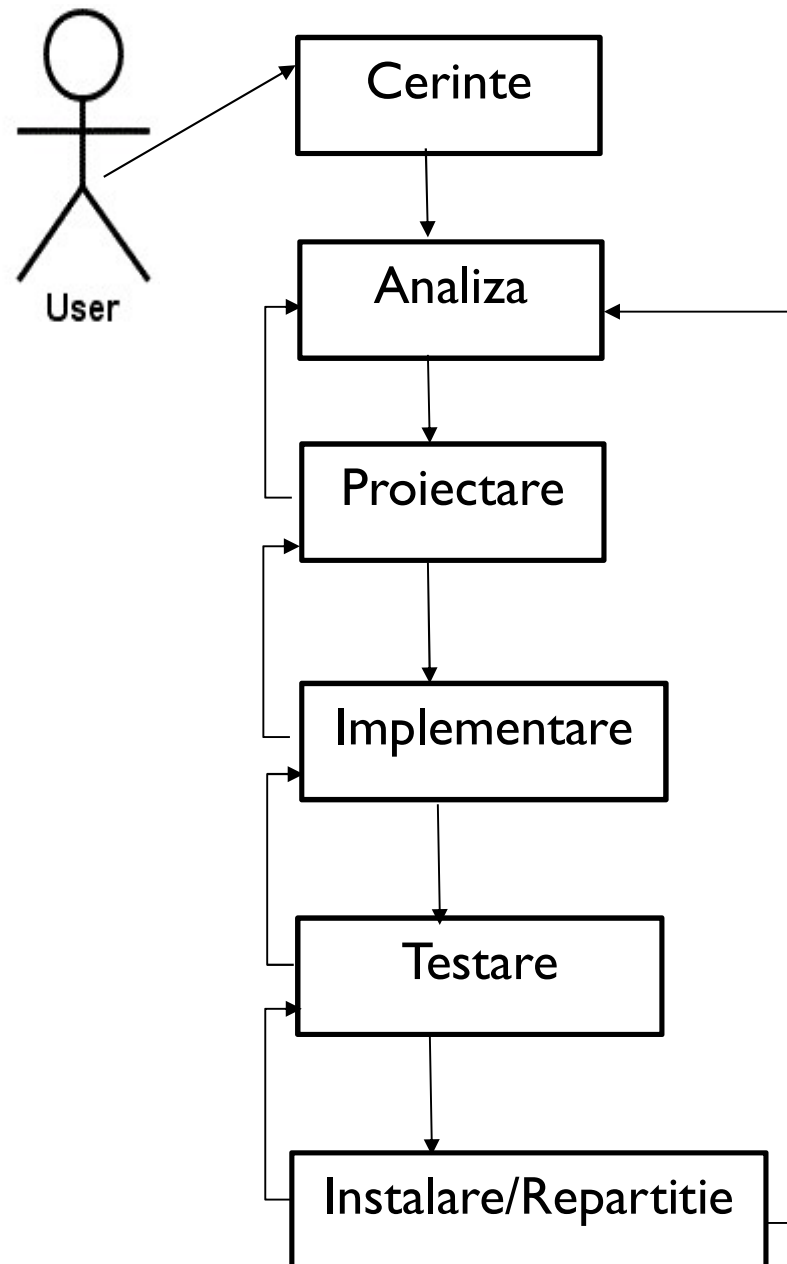


Model Driven Architecture

- MDA este o abordare suportată de OMG ce propune folosirea modelelor în dezvoltarea sistemelor și în special a sistemelor software.
- MDA propune construirea a trei vederi: modelul independent de calcul, modelul independent de platformă și modelul specific platformei.
- Modelul independent de calcul este o vedere a sistemului care nu arată detaliile de implementare ale sistemului. De obicei un astfel de model se mai numește modelul domeniului sau al afacerii.
- Modelul independent de platformă este o vedere a sistemului în care acesta este văzut ca o mașină virtuală (definită ca un sistem ce furnizează servicii) independentă de platformă.
- Modelul specific platformei îmbină modelul anterior cu detalii privind modul în care sistemul folosește un tip particular de platformă.



Activitatile procesului MDA

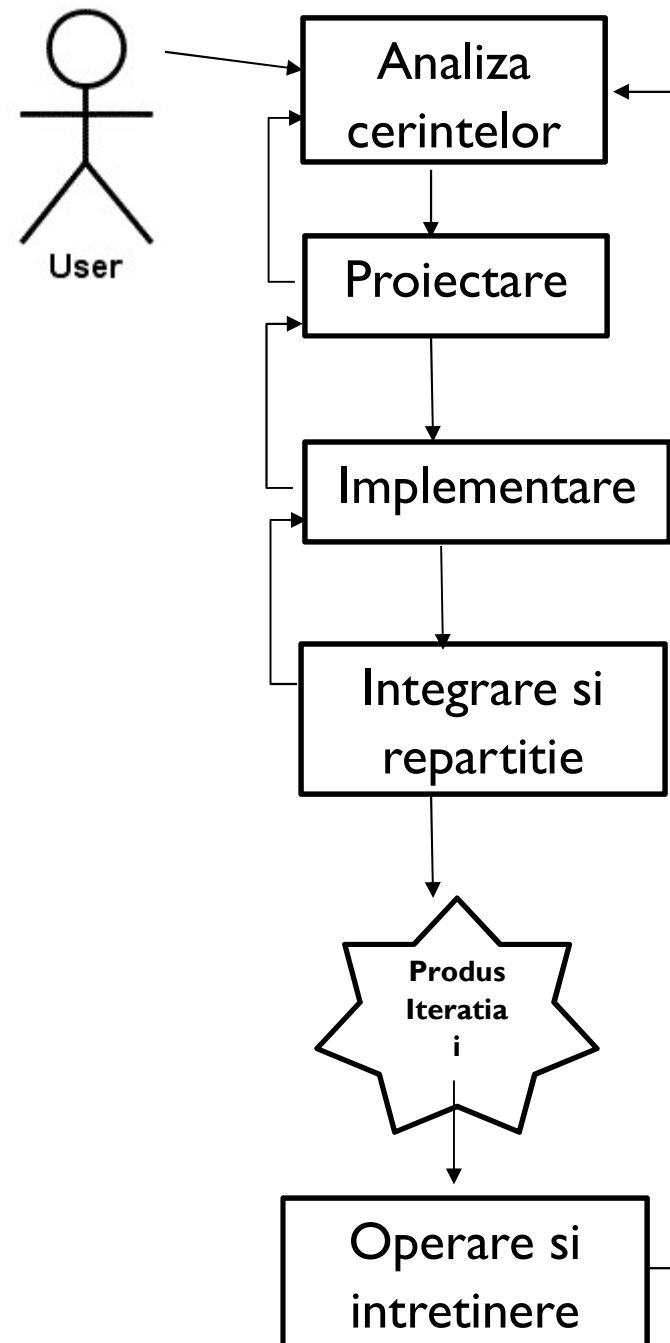


Dezvoltarea agila a software-ului

- Model propus in 2001 de catre organizatia non-profit Agile Alliance
- Principii:
 - Mai importanti sunt oamenii, decat procesele si instrumentele
 - Mai important este sa dezvoltam software, decat sa scriem documentatie
 - Mai important este sa colaboram cu clientii, decat sa negociem contracte
 - Mai important este sa modificam, decat sa urmam un plan
- Dezvoltare iterativa, condusa de testare, cu refactorizari frecvente, cu integrare continua, cu implicarea clientilor in procesul de dezvoltare
- Incurajeaza programarea in echipe de doi
- Roluri:
 - Client: ajuta la definirea produsului
 - Proiectant/Programator: ajuta la construirea produsului
 - Tester: verifica daca produsul functioneaza dupa cum a fost definit
 - Tracker : ajuta la strangerea si prezentarea metricilor
 - Instructor: instruieste si ghideaza echipa
 - Manager(optional): gestioneaza comunicarea externa



Activitatile dezvoltarii agile



Partea a III-a. Analiza cerintelor software



Modelul functional

- Caz de utilizare software = unitate coerenta de functionalitate furnizata de un clasificator manifestata prin secvente de mesaje intre clasificator si unul sau mai multi actori, impreuna cu actiunile realizate de sistem. Este un descriptor de comportament potential al unui clasificator in interactiunea lui cu actorii.
- Fiecare caz este descris printr-un sablon folosind limbaj natural care contine urmatoarele elemente:
 - nume ; descriere:
 - Actori software:
 - actori principali sau
 - actori secundari.
 - eveniment declansator;
 - preconditii si postconditii;
 - referinte incrucisate;
 - flux principal;
 - unul sau mai multe fluxuri alternative.



Aplicatia E-Biblioteca



Caz de utilizare: Creare cont

Nume: **Creare cont**

Descriere: Descrie comportamentul sistemului si interactiunea dintre bibliotecare si sistem pentru crearea unui cont nou

Actor software: Administrator/Bibliotecara (main)

Eveniment declansator: Administrator/Bibliotecara cere crearea unui cont nou

Preconditii: Sistemul trebuie sa functioneze corect

Postconditii: Sistemul a memorat datele contului creat si afisat profilul user

Referinte incrucisate:

Flux principal:

Administrator/Bibliotecara	Sistem
1. Cere crearea unui cont	2. Cere categorie utilizator (bibliotecar/administrator)
3. Trimite categorie (bibliotecar)	4. In functie de categoria utilizatorului, afiseaza un formular
5. Completeaza formularul si trimite datele cerute	6. Verifica datele primite (ID_angajat, username, parola) dpdv sintactic si semantic [A1]
	7. Creaza un cont nou
	8. Memoreaza datele contului creat
	9. Afiseaza profilul utilizatorului

Fluxuri alternative:

[A1]: Datele sunt incorecte

1. Sistemul afiseaza un mesaj de eroare
2. Fluxul continua cu pasul 4 din fluxul principal



Caz de utilizare: Login

Nume: **Login**

Descriere: Descrie comportamentul sistemului si interactiunea dintre useri si sistem pentru logarea unui utilizator

Actor software: User (main)

Eveniment declansator: User cere sa se logheze

Preconditii: Sistemul a memorat datele cel putin ale unui cont

Postconditii: Sistemul a afisat optiunile utilizatorului

Referinte incrucisate:

Flux principal:

User	Sistem
1. Cere sa se logheze	2. Cere username si parola
3. Trimite datele cerute [A1]	4. Verifica datele primite [A2]
	5. Afiseaza optiunile utilizatorului

Fluxuri alternative:

[A2] Nu trimite datele cerute

1. User anuleaza operatia
2. Sistemul reafiseaza pagina principala
3. Fluxul principal se termina.

[A1]: Datele sunt incorecte

1. Sistemul afiseaza un mesaj de eroare
2. Fluxul continua cu pasul 2 din fluxul principal

